COMS 4170 Software Testing Spring 2026

Exam II Review

1

Exam on Thursday April 28th

9:30-10:45

Carver 0205

Closed book

Can bring one page of 8.5 x11 inch notes (both sides)

No calculators needed

2

Material Covered

Chapters from Book

- *6.1 – Input Partitioning*
- *6.2-6.3 Combination Strategies*
- *7.1-7.3 Graph Coverage*
- *9.1-9.2 Syntax and Mutation Testing*

Other

- *Category Partition and Test Specification Language*
- *Metamorphic Testing*
- ~~*AI for Software Testing*~~
- *Homework 3 (CIT/TSL/Base Choice)*
- *Homework 4 (mutation testing and graphs)*

All material in the book is examinable, however an emphasis will be on material we have discussed in class

3

Types of Questions

- I may include a few short answer questions
- Most will expect you to work out a problem/think/connect the concepts together
- READ the questions completely. If I *ask for an explanation or example* – you will not get full credit without it

4

Input Space Partitioning

- Idea is that we can logically divide the input space into logical partitions of the input
- Independent of the RIPR model – focuses only on the input domain
- Input parameters:
  - Method parameters and non-local variables (unit testing), objects (integration testing), user inputs (system testing)

5

Input Domains

- The input domain for a program contains all the possible inputs to that program
- For even small programs, the input domain is so large that it might as well be infinite
- Testing is fundamentally about choosing finite sets of values from the input domain
- *Input parameters* define the scope of the input domain
  - Parameters to a method
  - Data read from a file
  - Global variables
  - User level inputs
- Input domains are partitioned into regions (blocks)
- At least one value is chosen from each block

6

Foundations

- Equivalence classes
  - All inputs in a class are expected to behave in the same way

- *Domain D*
- *Partition scheme q* of *D*
- The partition *q* defines a *set of blocks*, $Bq = b_1, b_2, \ldots, b_Q$

7

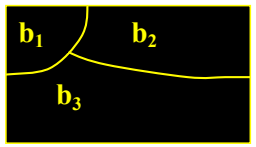
Foundations

- The partition must satisfy two properties:
  1. Blocks must be *pairwise disjoint* (no overlap)

$$b_i \cap b_j = \Phi, \forall\, i \neq j, b_i, b_j \in B_q$$

  2. Together the blocks *cover* the domain *D* (complete)

$$\bigcup_{b \in Bq} b = D$$



8

Using Partitions – Assumptions

- Choose a value from each block
- Each value is assumed to be equally useful for testing
- Application to testing
  - Find characteristics in the inputs : parameters, semantic descriptions, …
  - Partition each characteristic
  - Choose tests by combining values from characteristics
- Example Characteristics
  - Input X is null
  - Order of the input file F (sorted, inverse sorted, arbitrary, …)
  - Min separation of two aircraft
  - Input device (DVD, CD, VCR, computer, …)

9

Two Properties

- Blocks must not overlap (disjoint)
- Blocks cover the full input space (complete)

10

Two (separate) Parts to Modeling

1. Choose characteristics and partitions
2. Combining tests (coverage criteria)

11

Modeling the Input Domain

- Step 1 : Identify testable functions
  - Individual methods have one testable function
  - Methods in a class often have the same characteristics
  - Programs have more complicated characteristics—modeling documents such as UML can be used to design characteristics
  - Systems of integrated hardware and software components can use devices, operating systems, hardware platforms, browsers, etc.

12

Modeling the Input Domain

- Step 2: Find all the parameters
  - Often fairly straightforward, even mechanical
  - Important to be complete
  - Methods: Parameters and state (non-local) variables used
  - Components: Parameters to methods and state variables
  - System: All inputs, including files and databases

13

Modeling the Input Domain (*cont.*)

- Step 3: Model the input domain
  - The domain is scoped by the parameters
  - The structure is defined in terms of characteristics
  - Each characteristic is partitioned into sets of blocks
  - Each block represents a set of values
  - This is the *most creative design step in using ISP*

14

Modeling the Input Domain

Step 3: Model input domain

- Partitioning characteristics into blocks and values is a very creative engineering step
- More blocks means more tests
- Partitioning often flows directly from the definition of characteristics and both steps are done together
  - Should evaluate them separately – sometimes fewer characteristics can be used with more blocks and vice versa

15

Modeling the Input Domain

Step 3: Model input domain

Strategies for identifying values :

- Include valid, invalid and special values
- Sub-partition some blocks
- Explore boundaries of domains
- Include values that represent "normal use"
- Try to balance the number of blocks in each characteristic
- Check for completeness and disjointness!

16

Modeling the Input Domain

- Step 4 : Apply a test criterion to choose combinations of values
  - A test input has a value for each parameter
  - One block for each characteristic
  - Choosing all combinations is usually infeasible
  - Coverage criteria allow subsets to be chosen

17

Modeling the Input Domain

- Step 5 Refine combinations of blocks into test inputs
  - Choose appropriate values from each block

18

In Class Exercise

Unix Sort Utility

```
NAME
     sort -- sort or merge records (lines) of text and binary files

SYNOPSIS
     sort [-bcCdfghiRMmnrsuVz] [-k field1[,field2]] [-S memsize] [-T dir]
          [-t char] [-o output] [file ...]
     sort --help
     sort --version

DESCRIPTION
     The sort utility sorts text and binary files by lines.  A line is a record
     separated from the subsequent record by a newline (default) or NUL '\0'
     character (-z option).  A record can contain any printable or unprintable
     characters.  Comparisons are based on one or more sort keys extracted from
     each line of input, and are performed lexicographically, according to the
     current locale's collating rules and the specified command-line options
     that can tune the actual sorting behavior.  By default, if keys are not
     given, sort uses entire lines for comparison.

     The command line options are as follows:
```

19

Two Approaches to Input Domain Modeling

1. Interface-based approach
   - Develops characteristics directly from individual input parameters
   - Simplest application
   - Can be partially automated in some situations

20

Two Approaches to Input Domain Modeling

2. Functionality-based approach
   - Develops characteristics from a behavioral view of the program under test
   - Harder to develop—requires more design effort
   - May result in better tests, or fewer tests that are as effective

Input Domain Model (IDM)

Boundary Conditions

- We always need to consider the "boundaries" of a problem
- If we are testing something that cannot be zero, we need to test 0, 1, -1
- If we are searching for something in a using a list, we want to search the first, second, last, next to last position
- We place "boundaries" around any significant requirement

Functionality-Based *IDM—triang*()

- First two characterizations are based on syntax–parameters and their type
- A semantic level characterization could use the fact that the three

Geometric Characterization of *triang*()'s Inputs

Characteristic	b_1	b_2	b_3	b_4
q_1 = "Geometric Classification"	scalene	isosceles	equilateral	invalid

- Oops … something's fishy … equilateral is also isosceles !
- We need to refine the example to make characteristics valid

Correct Geometric Characterization of *triang*()'s Inputs

Characteristic	b_1	b_2	b_3	b_4
q_1 = "Geometric Classification"	scalene	isosceles, not equilateral	equilateral	invalid

Functionality-Based IDM—*triang*()

- A different approach would be to break the geometric characterization into four separate characteristics

Four Characteristics for *triang*()

Characteristic	b_1	b_2
q_1 = "Scalene"	True	False
q_2 = "Isosceles"	True	False
q_3 = "Equilateral"	True	False
q_4 = "Valid"	True	False

- Use constraints to ensure that
  - Equilateral = True implies Isosceles = True
  - Valid = False implies Scalene = Isosceles = Equilateral = False

Test Specification

- Consists of a list of categories
- Each category contains a list of choices
- The test specification used to create a set of test frames.
- Test frames are used to create actual test cases

25

25

TSL - Test Specification Language

- TSL - a means of implementing the category partition method
- If we write our test specifications using TSL we can develop test frames

26

26

S

```
#tsl for the example in class
Parameters:
        Pattern Size:
           empty.                              [property Empty]
           single character.                   [property NonEmpty]
           many character.                     [property NonEmpty]
           longer than any line in file.       [error]

        Quoting:
           pattern is quoted.                  [property Quoted]
           pattern is not quoted.              [if NonEmpty]
           pattern is improperly quoted.       [error]

        Embedded blanks:
           no embedded blankks.                [if NonEmpty]
           one embedded blank.                 [if NonEmpty && Quoted]
           several embedded blanks.            [if NonEmpty && Quoted]

        Embedded quotes:
           no embedded quotes.                 [if NonEmpty]
           one embedded quote.                 [if NonEmpty]
           several embedded quotes.            [if NonEmpty] [single]

        File name:
           good file name.
           no file with this name.             [error]
           omitted.                            [error]

Environments:

        Number of occurences of pattern in file:
           none.                               [if NonEmpty] [single]
           exactly one.                        [if NonEmpty] [property Match]
```

27

27

Special Constraints

Errors: if a choice always causes an error, it does not need to be tested in combination with other choices. We can use the [error] designation for this

Quoting:

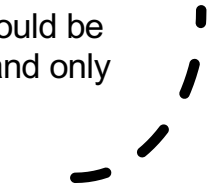
pattern is improperly quoted [error]

28

28

Special Constraints

- We can also say that a choice should not be combined with any other by using: [single]
- Often use this for a --help option or some other option that should be tested alone and only once.



29

29

Running the TSL Tool

```
mcohen>./tsl powerpoint.tsl
      27648 test frames generated and written to powerpoint.tsl.tsl
mcohen>
```

```
Test Case 1             (Key = 1.1.1.1.1.1.1.1.1.1.1.1.1.)
   Show_Extensions :  on
   Show_add-in     :  on
   Sound_feedback  :  on
   Open_gallery    :  on
   Turn_off_DM     :  on
   Print_Quality   :  high
   Designer        :  on
   Suggestions     :  on
   Alt_text        :  on
   Pen             :  on
   Macro_Security  :  disable_without
   Developer_Macro :  on
   Custom_Actions  :  do_not_allow


Test Case 2             (Key = 1.1.1.1.1.1.1.1.1.1.1.1.2.)
   Show_Extensions :  on
   Show_add-in     :  on
```

30

Step 4 – Choosing Combinations of Values (6.2)

- Once characteristics and partitions are defined, the next step is to choose test values
- We use criteria – to choose effective subsets

All Combinations (ACoC): All combinations of blocks from all characteristics must be used.

- Number of tests is the product of the number of blocks in each characteristic : $\prod_{i=1}^{Q}(B_i)$
- The second characterization of triang() results in 4*4*4 = 64 tests
  - Too many ?

31

ISP Criteria – All Combinations

- Consider the "second characterization" of Triang as given before:

Characteristic	b_1	b_2	b_3	b_4
q_1 = "Refinement of q_1"	greater than 1	equal to 1	equal to 0	less than 0
q_2 = "Refinement of q_2"	greater than 1	equal to 1	equal to 0	less than 0
q_3 = "Refinement of q_3"	greater than 1	equal to 1	equal to 0	less than 0

- For convenience, we relabel the blocks:

Characteristic	b_1	b_2	b_3	b_4
A	A1	A2	A3	A4
B	B1	B2	B3	B4
C	C1	C2	C3	C4

32

ISP Criteria – ACoC Tests

A1 B1 C1	A2 B1 C1	A3 B1 C1	A4 B1 C1
A1 B1 C2	A2 B1 C2	A3 B1 C2	A4 B1 C2
A1 B1 C3	A2 B1 C3	A3 B1 C3	A4 B1 C3
A1 B1 C4	A2 B1 C4	A3 B1 C4	A4 B1 C4
A1 B2 C1	A2 B2 C1	A3 B2 C1	A4 B2 C1
A1 B2 C2	A2 B2 C2	A3 B2 C2	A4 B2 C2
A1 B2 C3	A2 B2 C3	A3 B2 C3	A4 B2 C3
A1 B2 C4	A2 B2 C4	A3 B2 C4	A4 B2 C4
A1 B3 C1	A2 B3 C1	A3 B3 C1	A4 B3 C1
A1 B3 C2	A2 B3 C2	A3 B3 C2	A4 B3 C2
A1 B3 C3	A2 B3 C3	A3 B3 C3	A4 B3 C3
A1 B3 C4	A2 B3 C4	A3 B3 C4	A4 B3 C4
A1 B4 C1	A2 B4 C1	A3 B4 C1	A4 B4 C1
A1 B4 C2	A2 B4 C2	A3 B4 C2	A4 B4 C2
A1 B4 C3	A2 B4 C3	A3 B4 C3	A4 B4 C3
A1 B4 C4	A2 B4 C4	A3 B4 C4	A4 B4 C4

ACoC yields 4*4*4 = 64 tests for Triang!

This is almost certainly more than we need

Only 8 are valid (all sides greater than zero)

ISP Criteria – Each Choice

- 64 tests for triang() is almost certainly way too many
- Each Choice Coverage (ECC) : One value from each block for each characteristic must be used in at least one test case.
- should try at least one value from each block
- Number of tests is the number of blocks in the largest characteristic : $\text{Max}_{i=1}^{Q}(B_i)$

For *triang*() :	Substituting values:
A1, B1, C1	2, 2, 2
A2, B2, C2	1, 1, 1
A3, B3, C3	0, 0, 0
A4, B4, C4	-1,-1,-1

ISP Criteria – Pair-Wise

- Each choice yields few tests—cheap but maybe ineffective
- Pair-Wise Coverage (PWC) : A value from each block for each characteristic must be combined with a value from every block for each other characteristic.
- Number of tests is at least the product of two largest characteristics $(\text{Max}_{i=1}^{Q}(B_i)) * (\text{Max}_{j=1,\, j!=i}^{Q}(B_j))$

For *triang*() :			
A1, B1, C1	A1, B2, C2	A1, B3, C3	A1, B4, C4
A2, B1, C2	A2, B2, C3	A2, B3, C4	A2, B4, C1
A3, B1, C3	A3, B2, C4	A3, B3, C1	A3, B4, C2
A4, B1, C4	A4, B2, C1	A4, B3, C2	A4, B4, C3

ISP Criteria –T-Wise

- A natural extension is to require combinations

t-Wise Coverage (TWC) :A value from each block for each group of t characteristics must be combined.

- Number of tests is at least the product of *t* largest characteristics
- If all characteristics are the same size, the formula is

$$(\text{Max}_{i=1}^{Q}(B_i))^t$$

- If *t* is the number of characteristics *Q*, then all combinations
- That is … *Q-wise* = *AC*

ISP Criteria – Base Choice

- Testers sometimes recognize that certain values are important

Base Choice Coverage (BCC) : A base choice block is chosen for each characteristic, and a base test is formed by using the base choice for each characteristic. Subsequent tests are chosen by holding all but one base choice constant and using each non-base choice in each other characteristic.

- Number of tests is one base test + one test for each other block $1 + \sum_{i=1}^{Q} (B_i - 1)$

For *triang*() : Base	A1, B1, C1	A1, B1, C2	A1, B2, C1	A2, B1, C1
		A1, B1, C3	A1, B3, C1	A3, B1, C1
		A1, B1, C4	A1, B4, C1	A4, B1, C1

Base Choice simplified PowerPoint

```
Parameters:
        Open_gallery:
                on.
                off.
        Turn_off_DM:
                on.
                off.
        Print_Quality:
                high.
                med.
        Pen:
                on.
                off.
        Custom_Actions:
                do_not_allow.
                allow_with.
                allow_without.
```

Base:
OG: -on
Toff: -on
Print: med
Pen: off
Custom: don't allow

N=1+(1+1+1+1+2)=7

Base Choice Notes

- The base test must be feasible
  - That is, all base choices must be compatible
- Base choices can be
  - Most likely from an end-use point of view
  - Simplest
  - Smallest
  - First in some ordering
- Happy path tests often make good base choices
- The base choice is a crucial design decision
  - Test designers should document why the choices were made

ISP Criteria – Multiple Base Choice

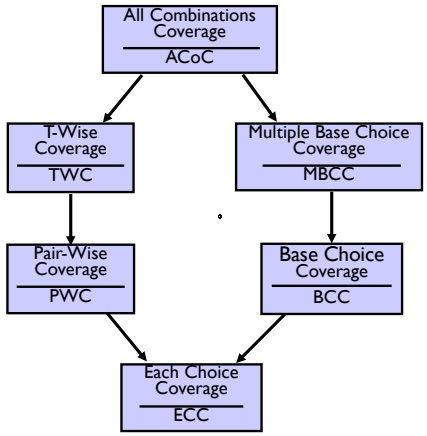
Multiple Base Choice Coverage (MBCC) : At least one, and possibly more, base choice blocks are chosen for each characteristic, and base tests are formed by using each base choice for each characteristic at least once. Subsequent tests are chosen by holding all but one base choice constant for each base test and using each non-base choice in each other characteristic.

- If M base tests and m_i base choices for each characteristic:

$$M + \sum_{i=1}^{Q} (M * (B_i - m_i))$$

For *triang*() : Bases			
A1, B1, C1	A1, B1, C3	A1, B3, C1	A3, B1, C1
	A1, B1, C4	A1, B4, C1	A4, B1, C1
A2, B2, C2	A2, B2, C3	A2, B3, C2	A3, B2, C2
	A2, B2, C4	A2, B4, C2	A4, B2, C2

ISP Coverage Criteria Subsumption



41

Combinatorial Interaction Testing (CIT)

- Test all *pairs* or *t*-way combinations of configuration options, where *t* is a defined strength of testing.

	Open Links In	Tab Bar	Select New Tabs From Links	Select New Tabs from history	When Closing Multiple Tabs
1	New Window	Hidden	Yes	Yes	Don't Warn
2	New Window	Visible	No	No	Warn
3	New Tab	Visible	No	Yes	Don't Warn
4	Most Recent	Hidden	No	Yes	Warn
5	New Tab	Hidden	Yes	No	Warn
6	Most Recent	Visible	Yes	No	Don't Warn

42

Mapping

```
#tsl for  powerpoint
Parameters:
        Show_Extensions:
                on.   0
                off.  1
        Show_add-in:
                on.
                off.
        Sound_feedback:
                on.
                off.
        Open_gallery:
                on.
                off.
        Turn_off_DM:
                on.
                off.
        Print_Quality:
                high.
                med.
                low.
        Designer:
                on.
                off.
        Suggestions:
                on.
                off.
        Alt_text:
                on.
                off.
        Pen:
                on.
                off.
```

```
t   k   v  TCount
2 13 29 387

 2 2 2 2 2 3 2 2 2 2 3 2 3

 0  1
 2  3
 4  5
 6  7
 8  9
 10  11  12
 13  14
 15  16
 17  18
 19  20
 21  22  23
 24  25
 26  27  28

10
on  0 2 4 7 9 12 13 16 17 19 23 25 27
    1 2 5 6 9 11 14 16 17 19 23 24 26
    0 2 5 7 9 10 14 16 17 19 22 25 26
    0 2 5 6 8 11 14 15 18 20 22 25 27
    1 3 4 6 9 12 13 16 18 20 22 24 28
    1 3 5 7 8 12 14 15 18 19 21 25 26
    0 3 4 7 9 11 13 15 17 20 21 24 26
    1 3 4 7 8 10 14 15 18 20 23 25 28
    1 3 5 6 8 10 13 16 18 19 21 24 27
    0 2 5 7 8 11 13 16 17 19 21 24 28
```

43

Creating a CIT Sample

```
2
13
2 5
3 1
2 4
3 1
2 1
3 1
```

```
t   k   v  TCount
2 13 29 387

 2 2 2 2 2 3 2 2 2 2 3 2 3

 0  1
 2  3
 4  5
 6  7
 8  9
 10  11  12
 13  14
 15  16
 17  18
 19  20
 21  22  23
 24  25
 26  27  28

10
 0 2 4 7 9 12 13 16 17 19 23 25 27
 1 2 5 6 9 11 14 16 17 19 23 24 26
 0 2 5 7 9 10 14 16 17 19 22 25 26
 0 2 5 6 8 11 14 15 18 20 22 25 27
 1 3 4 6 9 12 13 16 18 20 22 24 28
 1 3 5 7 8 12 14 15 18 19 21 25 26
 0 3 4 7 9 11 13 15 17 20 21 24 26
 1 3 4 7 8 10 14 15 18 20 23 25 28
 1 3 5 6 8 10 13 16 18 19 21 24 27
 0 2 5 7 8 11 13 16 17 19 21 24 28
```

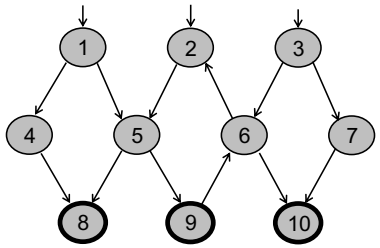
44

Definition of a Graph

- A set N of nodes, N is not empty
- A set N_0 of initial nodes, N_0 is not empty
- A set N_f of final nodes, N_f is not empty
- A set E of edges, each edge from one node to another
  - (n_i , n_j), i is predecessor, j is successor

Paths in Graphs

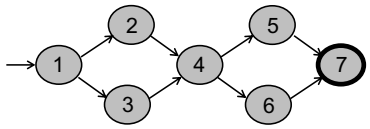
- Path: A sequence of nodes – [n_1, n_2, …, n_M]
  - Each pair of nodes is an edge
- Length: The number of edges
  - A single node is a path of length 0
- Subpath: A subsequence of nodes in p is a subpath of p



A Few Paths
[1, 4, 8]
[2, 5, 9, 6, 2]
[3, 7, 10]

Test Paths and SESEs

- Test Path: A path that starts at an initial node and ends at a final node
- Test paths represent execution of test cases
  - Some test paths can be executed by many tests
  - Some test paths cannot be executed by any tests
- SESE graphs : All test paths start at a single node and end at another node
  - Single-entry, single-exit (SESE)
  - N_0 and N_f have exactly one node



Double-diamond graph
Four test paths
[1, 2, 4, 5, 7]
[1, 2, 4, 6, 7]
[1, 3, 4, 5, 7]
[1, 3, 4, 6, 7]

Visiting and Touring

- Visit : A test path p *visits* node n if n is in p
  A test path p *visits* edge e if e is in p
- Tour : A test path p *tours* subpath q if q is a subpath of p

Path [1, 2, 4, 5, 7]
Visits nodes 1, 2, 4, 5, 7
Visits edges (1, 2), (2, 4), (4, 5), (5, 7)
Tours subpaths [1, 2, 4], [2, 4, 5], [4, 5, 7], [1, 2, 4, 5], [2, 4, 5, 7], [1, 2, 4, 5, 7]
(*Also, each edge is technically a subpath*)

Testing and Covering Graphs

- Test Requirements (TR): Describe properties of test paths
- Test Criterion : Rules that define test requirements
- Satisfaction: *Given a set TR of test requirements for a criterion C, a set of tests T satisfies C on a graph if and only if for every test requirement in TR, there is a test path in path(T) that meets the test requirement tr*
- Structural Coverage Criteria: Defined on a graph just in terms of nodes and edges
- Data Flow Coverage Criteria: Requires a graph to be annotated with references to variables

Node and Edge Coverage

- The first (and simplest) two criteria require that each node and edge in a graph be executed

Node Coverage (NC): Test set *T* satisfies node coverage on graph *G* iff for every syntactically reachable node *n* in *N*, there is some path *p* in *path(T)* such that *p* visits *n*.

- More simply stated:

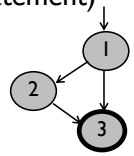
Node Coverage (NC) : TR contains each reachable node in G.

Node and Edge Coverage

- Edge coverage is slightly stronger than node

Edge Coverage (EC) : TR contains each reachable path of length up to 1, inclusive, in G.

- The phrase "*length up to 1*" allows for graphs with one node and no edges
- NC and EC are only different when there is an edge and another subpath between a pair of nodes (as in an "if-else" statement)



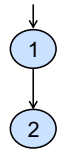
Node Coverage : TR = { 1, 2, 3 }
Test Path = [1, 2, 3]

Edge Coverage :TR = { (1, 2), (1, 3), (2, 3) }
Test Paths = [1, 2, 3]
[1, 3]

Paths of Length 1 and 0

- A graph with only one node will not have any edges

- It may seem trivial, but formally, Edge Coverage needs to require Node Coverage on this graph
- Otherwise, Edge Coverage will not subsume Node Coverage
  - So we define "length up to 1" instead of simply "length 1"
- We have the same issue with graphs that only have one edge – for Edge-Pair Coverage …

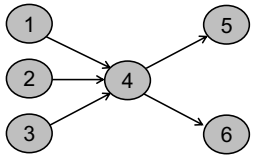


Covering Multiple Edges

- Edge-pair coverage requires pairs of edges, or subpaths of length 2

Edge-Pair Coverage (EPC) : TR contains each reachable path of length up to 2, inclusive, in G.

- The phrase "length up to 2" is used to include graphs that have less than 2 edges



Edge-Pair Coverage :

TR = { [1,4,5], [1,4,6], [2,4,5], [2,4,6], [3,4,5], [3,4,6] }

- The logical extension is to require all paths …

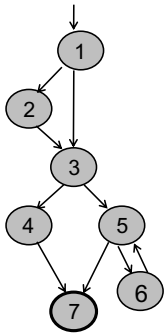
Covering Multiple Edges

Complete Path Coverage (CPC) :TR contains all paths in G.

Unfortunately, this is impossible if the graph has a loop, so a weak compromise makes the tester decide which paths:

Specified Path Coverage (SPC) : TR contains a set S of test paths, where S is supplied as a parameter.

Structural Coverage Example



Node Coverage
TR = { 1, 2, 3, 4, 5, 6, 7 }
Test Paths: [1, 2, 3, 4, 7] [1, 2, 3, 5, 6, 5, 7]

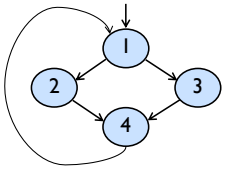
Edge Coverage
TR = { (1,2), (1, 3), (2, 3), (3, 4), (3, 5), (4, 7), (5, 6), (5, 7), (6, 5) }
Test Paths: [1, 2, 3, 4, 7] [1, 3, 5, 6, 5, 7]

Edge-Pair Coverage
TR = {[1,2,3], [1,3,4], [1,3,5], [2,3,4], [2,3,5], [3,4,7], [3,5,6], [3,5,7], [5,6,5], [6,5,6], [6,5,7] }
Test Paths: [1, 2, 3, 4, 7] [1, 2, 3, 5, 7] [1, 3, 4, 7] [1, 3, 5, 6, 5, 6, 5, 7]

Complete Path Coverage
Test Paths: [1, 2, 3, 4, 7] [1, 2, 3, 5, 7] [1, 2, 3, 5, 6, 5, 6] [1, 2, 3, 5, 6, 5, 6, 5, 7] [1, 2, 3, 5, 6, 5, 6, 5, 6, 5, 7] …

Simple Paths and Prime Paths

- Simple Path: *A path from node n_i to n_j is simple if no node appears more than once, except possibly the first and last nodes are the same*
  - No internal loops
  - A loop is a simple path
- Prime Path: *A simple path that does not appear as a proper subpath of any other simple path*



Simple Paths : [1,2,4,1], [1,3,4,1], [2,4,1,2], [2,4,1,3], [3,4,1,2], [3,4,1,3], [4,1,2,4], [4,1,3,4], [1,2,4], [1,3,4], [2,4,1], [3,4,1], [4,1,2], [4,1,3], [1,2], [1,3], [2,4], [3,4], [4,1], [1], [2], [3], [4]

Prime Paths : [2,4,1,2], [2,4,1,3], [1,3,4,1], [1,2,4,1], [3,4,1,2], [4,1,3,4], [4,1,2,4], [3,4,1,3]

Prime Path Coverage

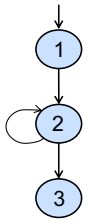
- A simple and finite criterion that requires loops to be executed as well as skipped

Prime Path Coverage (PPC) : TR contains each prime path in G.

- Will tour all paths of length 0, 1, …
- That is, it subsumes node and edge coverage
- PPC almost, but not quite, subsumes EPC …

PPC Does Not Subsume EPC

- If a node *n* has an edge to itself (*self edge*), EPC requires [*n, n, m*] and [*m, n, n*]
- [*n, n, m*] is not prime
- Neither [*n, n, m*] nor [*m, n, n*] are simple paths (not prime)



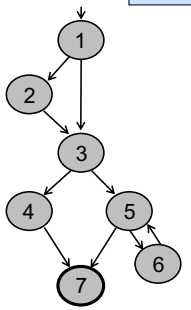
EPC Requirements :

TR = { [1,2,3], [1,2,2], [2,2,3], [2,2,2] }

PPC Requirements :

TR = { [1,2,3], [2,2] }

Simple & Prime Path Example



Simple paths

Len 0
[1]
[2]
[3]
[4]
[5]
[6]
[7] !

Len 1
[1, 2]
[1, 3]
[2, 3]
[3, 4]
[3, 5]
[4, 7] !
[5, 7] !
[5, 6]
[6, 5]

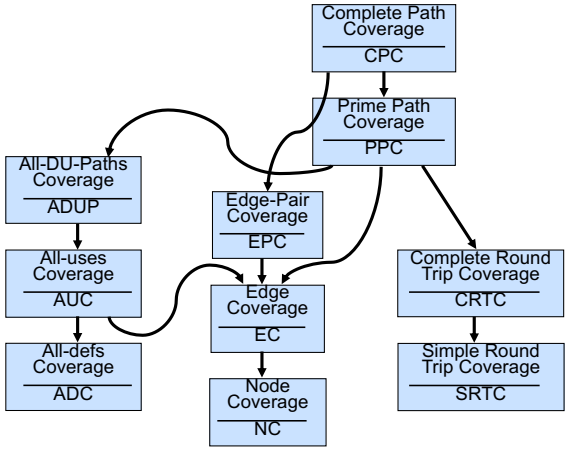
Len 2
[1, 2, 3]
[1, 3, 4]
[1, 3, 5]
[2, 3, 4]
[2, 3, 5]
[3, 4, 7] !
[3, 5, 7] !
[3, 5, 6] !
[5, 6, 5] *
[6, 5, 7] !
[6, 5, 6] *

Len 3
[1, 2, 3, 4]
[1, 2, 3, 5]
[1, 3, 4, 7] !
[1, 3, 5, 7] !
[1, 3, 5, 6] !
[2, 3, 4, 7] !
[2, 3, 5, 6] !
[2, 3, 5, 7] !

Len 4
[1, 2, 3, 4, 7] !
[1, 2, 3, 5, 7] !
[1, 2, 3, 5, 6] !

Prime Paths

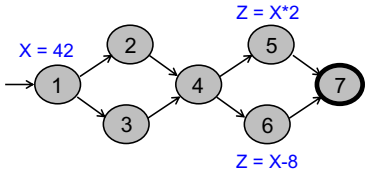
Graph Coverage Criteria Subsumption



Data Flow Criteria

Goal: Try to ensure that values are computed and used correctly

- Def: A location where a value for a variable is stored into memory
- Use: A location where a variable's value is accessed



Defs:	def (1) = {X}
	def (5) = {Z}
	def (6) = {Z}
Uses:	use (5) = {X}
	use (6) = {X}

The values given in defs should reach at least one, some, or all possible uses

61

DU Pairs and DU Paths

def (n) or def (e) : The set of variables that are defined by node n or edge e

use (n) or use (e) : The set of variables that are used by node n or edge e

DU (def-use) pair: A pair of locations (l_i, l_j) such that a variable v is defined at l_i and used at l_j

62

DU Pairs and DU Paths

Def-clear : A path from l_i to l_j is *def-clear* with respect to variable v if v is not given another value on any of the nodes or edges in the path

Reach : If there is a def-clear path from l_i to l_j with respect to v, the def of v at l_i reaches the use at l_j

du-path : A simple subpath that is def-clear with respect to v

from a def of v to a use of v

- du (n_i, n_j, v) – the set of du-paths from n_i to n_j
- du (n_i, v) – the set of du-paths that start at n_i

63

Control Flow Graphs

64

Overview

- A common application of graph criteria is to program source

- Graph : Usually the control flow graph (CFG)
- Node coverage : Execute every statement
- Edge coverage : Execute every branch
- Loops : Looping structures such as for loops, while loops, etc.

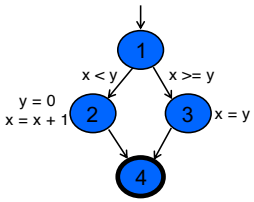
- Data flow coverage : Augment the CFG
  - defs are statements that assign values to variables
  - uses are statements that use variables

Control Flow Graphs

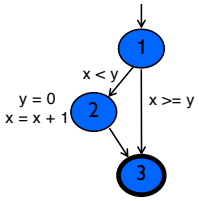
- A CFG models all executions of a method by describing control structures
- Nodes : Statements or sequences of statements (basic blocks)
- Edges : Transfers of control
- Basic Block : A sequence of statements such that if the first statement is executed, all statements will be (no branches)

CFG : The if Statement…

```
if (x < y)
{
  y = 0;
  x = x + 1;
}
else
{
  x = y;
}
```




```
if (x < y)
{
  y = 0;
  x = x + 1;
}
```

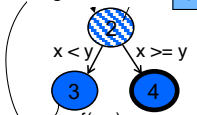


CFG : while and for Loops

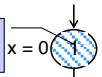
```
x = 0;
while (x < y)
{
  y = f (x, y);
  x = x + 1;
}
return (x);
```

Draw the graph and label the edges.

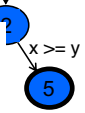
dummy node



implicitly initializes loop



Draw the graph and label the edges.

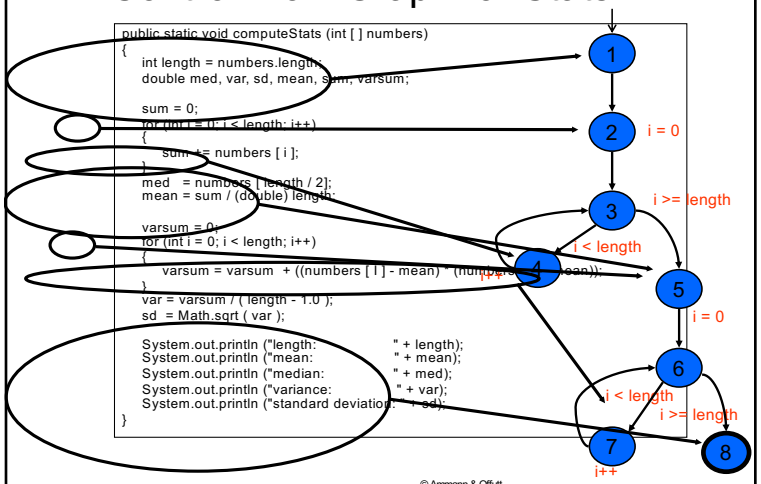



```
for (x = 0; x < y; x++)
{
  y = f (x, y);
}
return (x);
```

implicitly increments loop

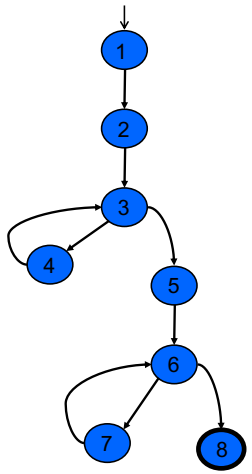
Control Flow Graph for Stats

```
public static void computeStats (int [ ] numbers)
{
    int length = numbers.length;
    double med, var, sd, mean, sum, varsum;

    sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += numbers [ i ];
    }
    med   = numbers [ length / 2];
    mean = sum / (double) length;

    varsum = 0;
    for (int i = 0; i < length; i++)
    {
        varsum = varsum  + ((numbers [ i ] - mean) * (numbers [ i ] - mean));
    }
    var = varsum / ( length - 1.0 );
    sd  = Math.sqrt ( var );

    System.out.println ("length:              " + length);
    System.out.println ("mean:                " + mean);
    System.out.println ("median:              " + med);
    System.out.println ("variance:            " + var);
    System.out.println ("standard deviation:  " + sd);
}
```



69

Control Flow TRs and Test Paths—EC



Edge Coverage

TR	Test Path
A. [1, 2] B. [2, 3] C. [3, 4] D. [3, 5] E. [4, 3] F. [5, 6] G. [6, 7] H. [6, 8] I. [7, 6]	[1, 2, 3, 4, 3, 5, 6, 7, 6, 8]

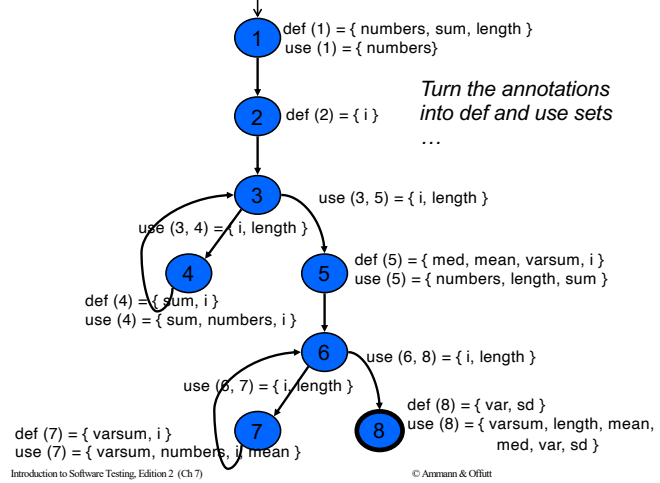
70

Data Flow Coverage for Source

- use : a location where variable's value is accessed
  - x appears on the right side of an assignment
  - x appears in a conditional test
  - x is an actual parameter to a method
  - x is an output of the program
  - x is an output of a method in a return statement
- If a def and a use appear on the same node, then it is only a DU-pair if the def occurs after the use and the node is in a loop

71

CFG for Stats – With Defs & Uses



72

Defs and Uses Tables for Stats

Node	Def	Use
1	{ numbers, sum, length }	{ numbers }
2	{ i }	
3		
4	{ sum, i }	{ numbers, i, sum }
5	{ med, mean, varsum, i }	{ numbers, length, sum }
6		
7	{ varsum, i }	{ varsum, numbers, i, mean }
8	{ var, sd }	{ varsum, length, var, mean, med, var, sd }

Edge	Use
(1, 2)	
(2, 3)	
(3, 4)	{ i, length }
(4, 3)	
(3, 5)	{ i, length }
(5, 6)	
(6, 7)	{ i, length }
(7, 6)	
(6, 8)	{ i, length }

73

DU Pairs for Stats

variable	DU Pairs
numbers	(1, 4) (1, 5) (1, 7)
length	(1, 5) (1, 8) (1, (3,4)) (1, (3,5)) (1, (6,7)) (1, (6,8))
med	(5, 8)
var	(8, 8)
sd	(8, 8)
mean	(5, 7) (5, 8)
sum	(1, 4) (1, 5) (4, 4) (4, 5)
varsum	(5, 7) (5, 8) (7, 7) (7, 8)
i	(2, 4) (2, (3,4)) (2, (3,5)) (2, 7) (2, (6,7)) (2, (6,8)) (4, 4) (4, (3,4)) (4, (3,5)) (4, 7) (4, (6,7)) (4, (6,8)) (5, 7) (5, (6,7)) (5, (6,8)) (7, 7) (7, (6,7)) (7, (6,8))

defs come before uses, do not count as DU pairs
These are local uses

defs after use in loop, these are valid DU pairs

No def-clear path … different scope for i

No path through graph from nodes 5 and 7 to 4 or 3

74

Grammar Coverage Criteria

- Software engineering makes practical use of automata theory in several ways
  - Programming languages defined in BNF
  - Program behavior described as finite state machines
  - Allowable inputs defined by grammars
- A simple regular expression:

*(G s n | B t n)**

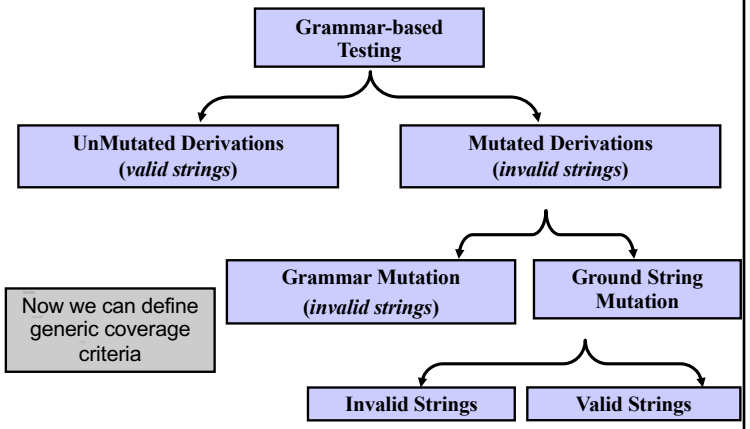
'*' is *closure* operator, zero or more occurrences

'|' is *choice*, either one can be used

- Any sequence of "*G s n*" and "*B t n*"
- '*G*' and '*B*' could represent commands, methods, or events
- '*s*', '*t*', and '*n*' can represent arguments, parameters, or values
- '*s*', '*t*', and '*n*' could represent literals or a set of values

75

Mutation as Grammar-Based Testing



76

Grammar-based Coverage Criteria (9.1.1)

- The most common and straightforward criteria use every terminal and every production at least once

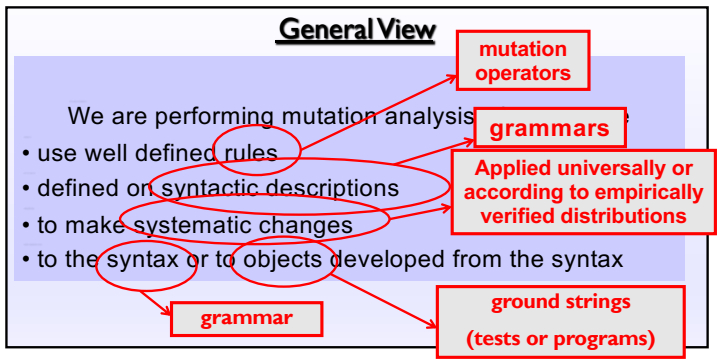
Terminal Symbol Coverage (TSC) : TR contains each terminal symbol *t* in the grammar *G*.

Production Coverage (PDC) : TR contains each production *p* in the grammar *G*.

Mutation Testing (9.1.2)

- Grammars describe both valid and invalid strings
- Both types can be produced as mutants
- A mutant is a variation of a valid string
  - Mutants may be valid or invalid strings
- Mutation is based on "mutation operators" and "ground strings"

What is Mutation ?



Mutation Testing

- Ground string: A string in the grammar
  - The term "ground" is used as an analogy to algebraic ground terms
- Mutation Operator : A rule that specifies syntactic variations of strings generated from a grammar
- Mutant: The result of one application of a mutation operator
  - A mutant is a string either in the grammar or very close to being in the grammar

What is a Mutant?

Mutant

A small syntactic change to a programming artifact

(program, statechart, XML, SQL, specification, …)

Mutation operators are defined on the underlying grammar

(change an operator to another compatible operator, change an edge from one target state to another, …)

Killing Mutants

- When ground strings are mutated to create valid strings, the hope is to exhibit different behavior from the ground string
- This is normally used when the grammars are programming languages, the strings are programs, and the ground strings are pre-existing programs

Killing Mutants

- Killing Mutants: Given a mutant $m \in M$ for a derivation D and a test t, t is said to kill m if and only if the output of t on D is different from the output of t on m
- The derivation D may be represented by the list of productions or by the final string

Syntax-based Coverage Criteria

- Coverage is defined in terms of killing mutants

Mutation Coverage (MC) : For each $m \in M$, TR contains exactly one requirement, to kill m.

- Coverage in mutation equates to number of mutants killed
- The ratio of mutants killed is called the mutation score

Syntax-based Coverage Criteria

- When creating invalid strings, we just apply the operators
- This results in two simple criteria
- It makes sense to either use every operator once or

Mutation Operator Coverage (MOC) : For each mutation operator, TR contains exactly one requirement, to create a mutated string *m* that is derived using the mutation operator.

Mutation Production Coverage (MPC) : For each mutation operator, TR contains several requirements, to create one mutated string *m* that includes every production that can be mutated by that operator.

Categorizing Mutants

Testers can keep adding tests until all mutants have been killed

Trivial mutant : Almost every test can kill it

Equivalent mutant : No test can kill it (same behavior as original)

Program Mutation Example

Original Method

```
int Min (int A, int B)
{
      int minVal;
      minVal = A;
      if (B < A)
      {
          minVal = B;
       }
       return (minVal);
} // end Min
```

6 mutants

Each represents a separate program

With Embedded Mutants

```
int Min (int A, int B)
{
      int minVal;
      minVal = A;
Δ 1   minVal = B;
      if (B < A)
Δ 2   if (B >= A)
Δ 3   if (B < minVal)
      {
            minVal = B;
Δ 4         Bomb ();
Δ 5         minVal = A;
Δ 6         minVal = failOnZero (B);
      }
      return (minVal);
} // end Min
```

Replace one variable with another

Replaces operator

Immediate runtime failure … if reached

Immediate runtime failure if B==0, else does nothing

Weak Mutation Example

- Mutant 1 in the Min( ) example is:

```
      minVal = A;
Δ 1   minVal = B;
      if (B < A)
          minVal = B;
```

- The complete test specification to kill mutant 1:

- Reachability : *true* // Always get to that statement
- Infection : $A \neq B$
- Propagation: $(B < A) = false$ // Skip the next assignment
- Full Test Specification : $true \wedge (A \neq B) \wedge ((B < A) = false)$
  $\equiv (A \neq B) \wedge (B \geq A)$
  $\equiv (B > A)$
- Weakly kill mutant 1, but not strongly? A = 5, B = 3

Mutation Operators for muJava

1. ABS — Absolute Value Insertion
2. AOR — Arithmetic Operator Replacement
3. ROR — Relational Operator Replacement
4. COR — Conditional Operator Replacement
5. SOR — Shift Operator Replacement
6. LOR — Logical Operator Replacement
7. ASR — Assignment Operator Replacement
8. UOI — Unary Operator Insertion
9. UOD — Unary Operator Deletion
10. SVR — Scalar Variable Replacement
11. BSR — Bomb Statement Replacement

PITest

Triangle.java

```
1  package com.example;
2
3
4  public class Triangle {
5
6    public static TriangleType classify(final int a, final int b, final int c) {
7      int trian;
8  4   if ((a <= 0) || (b <= 0) || (c <= 0)) {
9  1     return TriangleType.INVALID;
10     }
11     trian = 0;
12 2   if (a == b) {
13 1     trian = trian + 1;
14     }
15 2   if (a == c) {
16 1     trian = trian + 2;
17     }
18 2   if (b == c) {
19 1     trian = trian + 3;
20     }
21 2   if (trian == 0) {
22 1     if (((a + b) < c) || ((a + c) < b) || ((b +
23 1       return TriangleType.INVALID;
24       } else {
25 1       return TriangleType.SCALENE;
26       }
27     }
28 2   if (trian > 3) {
```

Pit Test Coverage Report

Package Summary

com.example

Number of Classes	Line Coverage		Mutation Coverage	
1	74%	17/23	57%	29/51

Breakdown by Class

Name	Line Coverage		Mutation Coverage	
Triangle.java	74%	17/23	57%	29/51

Report generated by PIT 1.4.7

FEATURE: AUTOMATED SOFTWARE TESTING

Metamorphic Testing: Testing the Untestable

University of Seville

and the programs suffering fr
are often referred to as *nontes*
(or *untestable*).[1–3]

Metamorphic testing (MT)
effective technique for alleviatir
oracle problem and, thus, for t
untestable programs where, as
previous example, failure
vealed by che

Reading

Segura, S., Towey, D., Zhou, Z.Q., Chen, T.Y., Testing: Testing the Untestable, *IEEE Software,* April, 2020

Example

- ***What we do know:***
  - If we perform merge(L_1, L_2) and then perform merge (L_2,L_1) the order of merging should not change the outcome

Metamorphic relationship:
merge(L_1, L_2) = merge(L_2,L_1)

Merge(L1,L2) → source test case
Merge(L2,L1)→ follow up test case

Example

- ***What we do know:***
  - If we perform merge(L_1, L_2) and then perform merge (L_2,L_1) the order of merging should not change the outcome

Metamorphic relationship:
merge(L_1, L_2) = merge(L_2,L_1)

Merge(L1,L2) → source test case
Merge(L2,L1)→ follow up test case

93

Case Study

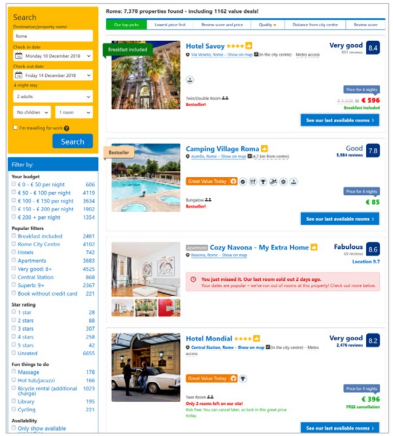


Figure 1. Booking.com search interface.

94

Metamorphic Relations

MR_1 : Perform search, then perform a search with a budget filter (US $100-150) Result of second should be subset of first

MR_2: Perform search for "1 star" hotels. Then repeat 4 times changing to 2, 3, 4, and 5 stars

Results of all 5 should be disjoint

MR_3:Perform a search. Then repeat changing ordering from "Our top picks" to "Review score"

Both should return the same items, regardless of their ordering

95